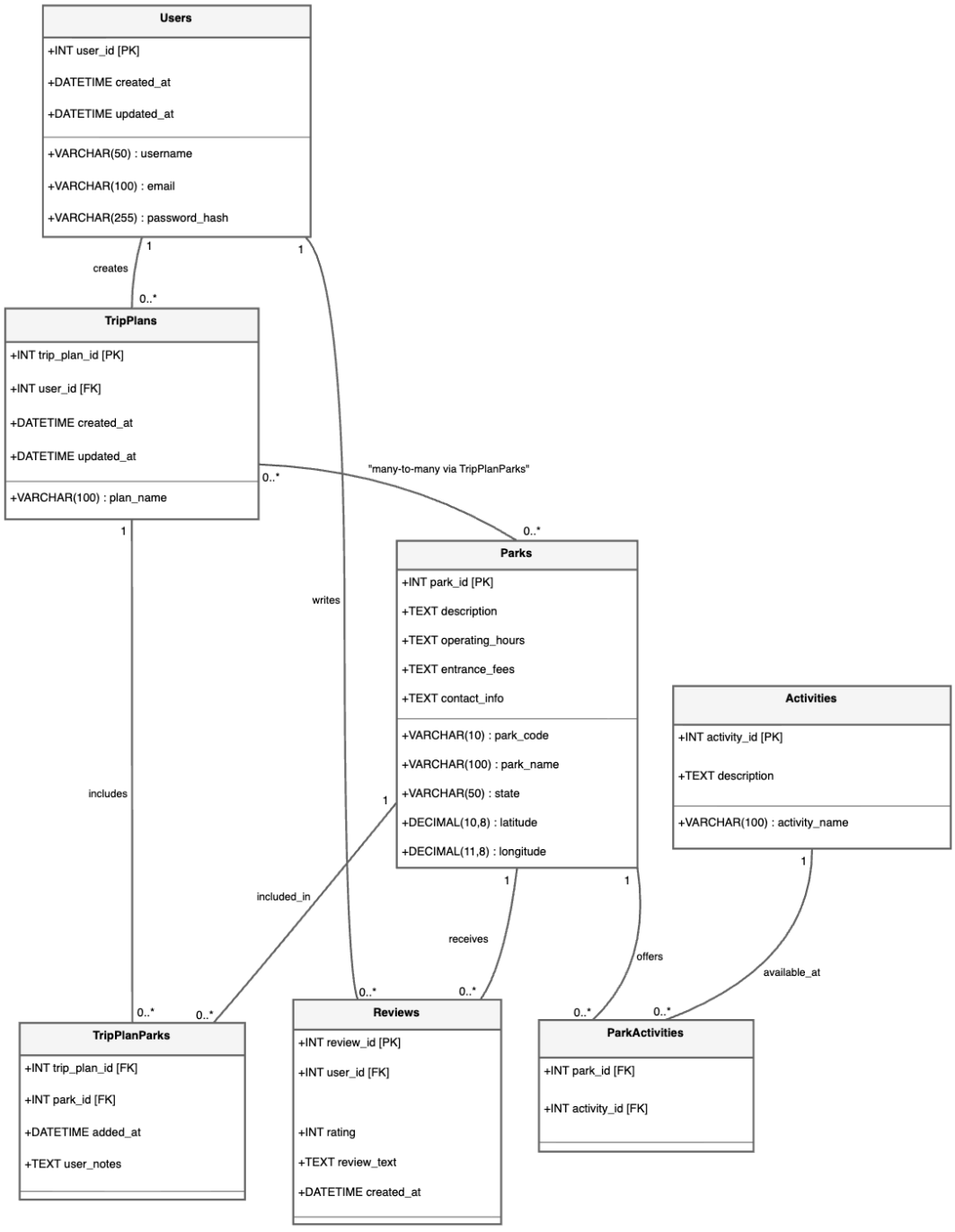
PT1-Stage 2: Conceptual and Logical Database Design

Terrain: National Park Adventure Planner

UML Class Diagram:

Link:https://drive.google.com/file/d/1aLu1mpfOtW7LUCbbAYx5DgjIQMYnvZjo/view?usp=sharing



UML Explanation:

Entities Description

1) **Users**

   Purpose: Represents all users of the Terrain platform who create accounts to plan National Park trips.

   Attributes:

   - user_id (Primary Key): Unique identifier for each user
   - username: Display name chosen by user
   - email: User's email address for authentication and communication
   - password_hash: Securely hashed password for authentication
   - created_at: Timestamp when account was created
   - updated_at: Timestamp when account was last modified

   Assumptions: Each user has a unique identifier and email address. User authentication is handled securely with hashed passwords. Users can create multiple trip plans and write multiple reviews.

2) **Parks**

   Purpose**:** Represents National Parks with comprehensive information sourced from the National Park Service API.

   Attributes**:**

   - park_id (Primary Key): Unique identifier for each park
   - park_code: Official NPS park code (example, "YELL" for Yellowstone)
   - park_name: Full name of the National Park
   - description: Detailed description of the park
   - state: State(s) where the park is located
   - latitude: Geographic latitude coordinate
   - longitude: Geographic longitude coordinate
   - operating_hours: Current operating hours information
   - entrance_fees: Fee structure information
   - contact_info: Contact information for the park

   Assumptions**:** Park data is primarily sourced from NPS API and updated regularly. Each park has unique identifiers and geographic coordinates. Parks can be included in multiple trip plans and have multiple reviews.

3) **TripPlans**

Purpose: Represents user created trip plans that can contain multiple parks.

Attributes:

- trip_plan_id (Primary Key): Unique identifier for each trip plan
- user_id (Foreign Key): References the user who created the plan
- plan_name: User defined name for the trip plan
- created_at: Timestamp when plan was created
- updated_at: Timestamp when plan was last modified

Assumptions: Each trip plan belongs to exactly one user. Users can create multiple trip plans. Trip plans can contain multiple parks through the TripPlanParks junction table.

4) TripPlanParks

Purpose: Represents the many to many relationship between TripPlans and Parks, allowing multiple parks per trip, and each park to be included in multiple trips.

Attributes:

- trip_plan_id (FK): References TripPlan
- park_id (FK): References Park
- added_at: Date park added to trip
- user_notes: Notes for that park in that trip

Assumptions: Many to many enables users to add any number of parks to plans, and parks can appear in many plans.

5) Reviews

Purpose: Represents user reviews and ratings for National Parks.

Attributes:

- review_id (Primary Key): Unique identifier for each review
- user_id (Foreign Key): References the user who wrote the review
- park_id (Foreign Key): References the park being reviewed
- rating: Numeric rating (typically 1-5 scale)
- review_text: Written review content
- created_at: Timestamp when review was created

Assumptions: Each review is written by one user about one park. Users can write multiple reviews, and parks can have multiple reviews. Reviews help other users make informed decisions about park visits.

6) Activities

Purpose: Represents various activities available at National Parks (hiking, camping, wildlife viewing, etc.).

Attributes:

- activity_id (Primary Key): Unique identifier for each activity type
- activity_name: Name of the activity
- description: Detailed description of the activity

Assumptions: Activities are standardized across parks but may not be available at every park. This entity allows for flexible activity management and filtering capabilities.

7) ParkActivities

Purpose: Junction table that establishes many to many relationship between parks and activities.

Attributes:

- park_id (Foreign Key): References the park
- activity_id (Foreign Key): References the activity

Assumptions: Parks can offer multiple activities, and activities can be available at multiple parks. This relationship allows users to search for parks based on desired activities.

Relationships and Cardinality

1. Users to TripPlans (One-to-Many)
    - Relationship: A user can create multiple trip plans (1..*), but each trip plan belongs to exactly one user (1..1).
    - Explanation: Users need the ability to organize their park visits into separate trip plans (e.g., "Summer 2024 Western Parks", "Family Vacation Plan"). Each plan is owned by a single user to maintain privacy and organization.
2. Users to Reviews (One-to-Many)
    - Relationship: A user can write multiple reviews (0..*), but each review is written by exactly one user (1..1).
    - Explanation: Users can share their experiences by reviewing parks they've visited. Each review must be attributed to a specific user for credibility and accountability.
3. TripPlans to Parks (Many-to-Many)
    - Relationship: Each TripPlan can include multiple Parks (0..), and each Park can be part of multiple TripPlans (0..). This relationship is implemented via the TripPlanParks junction table.

    - Explanation: A trip often consists of visits to several parks, and parks are visited as part of multiple different trips by various users.
4. Parks to Activities (Many-to-Many)
    - Relationship: Parks can offer multiple activities (0..), and activities can be available at multiple parks (0..). This relationship is implemented with ParkActivities junction table.
    - Explanation: National Parks offer diverse recreational opportunities. The junction table captures which specific activities are available at each park and supports activity and park searches and filtering.
5. Parks to Reviews (One-to-Many)
    - Relationship: A park can have multiple reviews (0..*), but each review is about exactly one park (1..1).
    - Explanation: Parks benefit from multiple user perspectives through reviews. Each review focuses on a specific park to maintain relevance and clarity.
6. Parks to ParkActivities (One-to-Many)
    - Relationship: A park can offer multiple activities (0..*), and each ParkActivities entry relates to exactly one park (1..1).
    - Explanation: National Parks typically offer diverse recreational opportunities. The junction table captures which specific activities are available at each park.
7. Activities to ParkActivities (One-to-Many)
    - Relationship: An activity can be available at multiple parks (0..*), and each ParkActivities entry references exactly one activity (1..1).
    - Explanation: Common activities like hiking or camping are available at many parks. This relationship supports activity based park searches and filtering.

This database design satisfies the requirements for multiple types of relationships by implementing:

- One-to-Many: Users → TripPlans, Users → Reviews, Parks → Reviews, Parks → ParkActivities, Activities → ParkActivities
- Many-to-Many: TripPlans ↔ Parks (via TripPlanParks), Parks ↔ Activities (via ParkActivities)

Why Model These as Entities Instead of Attributes?

- Users are modeled as a separate entity because they represent independent actors in the system with their own lifecycle, authentication requirements, and relationships to multiple other entities.
- Parks are the central subject matter of the application and contain complex, multi source data (NPS API, weather API, air quality API). They require independent management and have relationships with multiple entities.
- Trip plans represent user generated organizational structures that exist independently of other entities. They require their own metadata (creation dates, names) and support many to many relationships with parks.

- Reviews are substantial user generated content that require attribution, timestamps, and independent management. They support community features and decision making processes.
- Activities are standardized entities that exist across multiple parks and support search/filtering functionality. They require independent description and management.
- Junction tables (TripPlanParks, ParkActivities) are modeled as entities rather than simple relationships because they contain additional attributes (added_at, user_notes for TripPlanParks) and support complex querying requirements.

Relational Schema

Users(user_id: INT [PK], username: VARCHAR(50), email: VARCHAR(100), password_hash: VARCHAR(255), created_at: DATETIME, updated_at: DATETIME)

Parks(park_id: INT [PK], park_code: VARCHAR(10), park_name: VARCHAR(100), description: TEXT, state: VARCHAR(50), latitude: DECIMAL(10,8), longitude: DECIMAL(11,8), operating_hours: TEXT, entrance_fees: TEXT, contact_info: TEXT)

TripPlans(trip_plan_id: INT [PK], user_id: INT [FK to Users.user_id], plan_name: VARCHAR(100), created_at: DATETIME, updated_at: DATETIME)

TripPlanParks(trip_plan_id: INT [FK to TripPlans.trip_plan_id], park_id: INT [FK to Parks.park_id], added_at: DATETIME, user_notes: TEXT, [PK(trip_plan_id, park_id)])

Reviews(review_id: INT [PK], user_id: INT [FK to Users.user_id], park_id: INT [FK to Parks.park_id], rating: INT, review_text: TEXT, created_at: DATETIME)

Activities(activity_id: INT [PK], activity_name: VARCHAR(100), description: TEXT)

ParkActivities(park_id: INT [FK to Parks.park_id], activity_id: INT [FK to Activities.activity_id], [PK(park_id, activity_id)])

Normalization Process

To ensure the Terrain database is properly normalized, we have verified that it satisfies Third Normal Form (3NF), which requires that all non key attributes are fully functionally dependent on the primary key and no transitive dependencies exist.

Users Table

- Primary Key: user_id
- Functional Dependencies: user_id → {username, email, password_hash, created_at, updated_at}

- 3NF Compliance: All non key attributes are directly dependent on user_id with no transitive dependencies.

Parks Table

- Primary Key: park_id
- Functional Dependencies: park_id → {park_code, park_name, description, state, latitude, longitude, operating_hours, entrance_fees, contact_info}
- 3NF Compliance: All park related attributes are directly determined by park_id. Geographic and operational data belong together as they describe park characteristics.

TripPlans Table

- Primary Key: trip_plan_id
- Functional Dependencies: trip_plan_id → {user_id, plan_name, created_at, updated_at}
- 3NF Compliance: All attributes are directly dependent on trip_plan_id. The user_id is a foreign key reference, not a transitive dependency.

TripPlanParks Table

- Primary Key: (trip_plan_id, park_id) - Composite key
- Functional Dependencies: (trip_plan_id, park_id) → {added_at, user_notes}
- 3NF Compliance: Non key attributes are dependent on the full composite key, representing relationship specific data.

Reviews Table

- Primary Key: review_id
- Functional Dependencies: review_id → {user_id, park_id, rating, review_text, created_at}
- 3NF Compliance: All attributes are directly dependent on review_id. Foreign keys reference independent entities.

Activities Table

- Primary Key: activity_id
- Functional Dependencies: activity_id → {activity_name, description}
- 3NF Compliance: Both attributes are directly determined by activity_id with no redundancy.

ParkActivities Table

- Primary Key: (park_id, activity_id) - Composite key
- Functional Dependencies: None (only foreign keys)
  3NF Compliance: Pure junction table with no additional attributes that could create dependencies.

Additional Normalization Considerations:

- No repeating groups: Every column holds a single, atomic value, no multi valued fields or nested lists.
- Full functional dependency: In tables with composite primary keys, each non key attribute depends on the entire key, not just a piece of it.
- No transitive dependencies: Non key attributes point directly to the primary key, they don't depend on other non key attributes.
- Foreign key integrity: Every foreign key is enforced and references a valid primary key in the related table.
- 3NF achieved: With the above in place, the schema meets Third Normal Form, reducing redundancy while keeping queries fast and the data consistent for Terrain.